



Primera Llamada al Modelo

Redes neuronales, modelos LLM y roles del sistema

Contenido de la clase

Lo que vamos a cubrir hoy

1

¿Qué son las redes neuronales?

2

¿Qué es un modelo de IA?

3

Roles del sistema (system, user, assistant)

4

Parámetros de configuración

5

Modelo y estructura de costos

6

Capa gratuita y API key

¿Qué son las redes neuronales?

Los modelos de IA (LLM) están contruidos sobre esta idea

1

Inspiradas en el cerebro

Los LLM son redes neuronales, inspiradas en el funcionamiento del cerebro humano.

2

Nodos conectados

Formadas por neuronas artificiales conectadas entre sí, que transmiten señales de entrada a salida.

3

Organizadas en capas

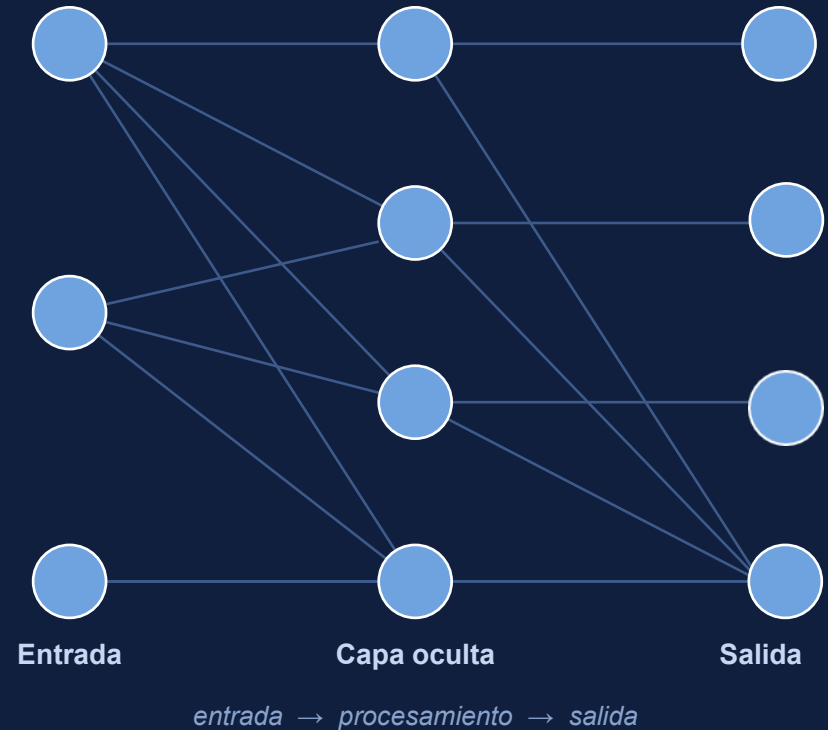
Las entradas llegan a neuronas agrupadas en capas, que en conjunto forman la red.

4

Aprenden solas

Aprenden a modificarse automáticamente para resolver tareas que las reglas clásicas no pueden.

Ejemplo visual



¿Qué es un modelo de IA?

Una red neuronal ya entrenada, lista para usarse

1

Parte de una red neuronal

Un modelo de IA es una red neuronal entrenada con grandes cantidades de datos.

2

Ajusta sus pesos

Durante el entrenamiento, la red modifica automáticamente sus conexiones para aprender patrones.

3

Red + conocimiento aprendido

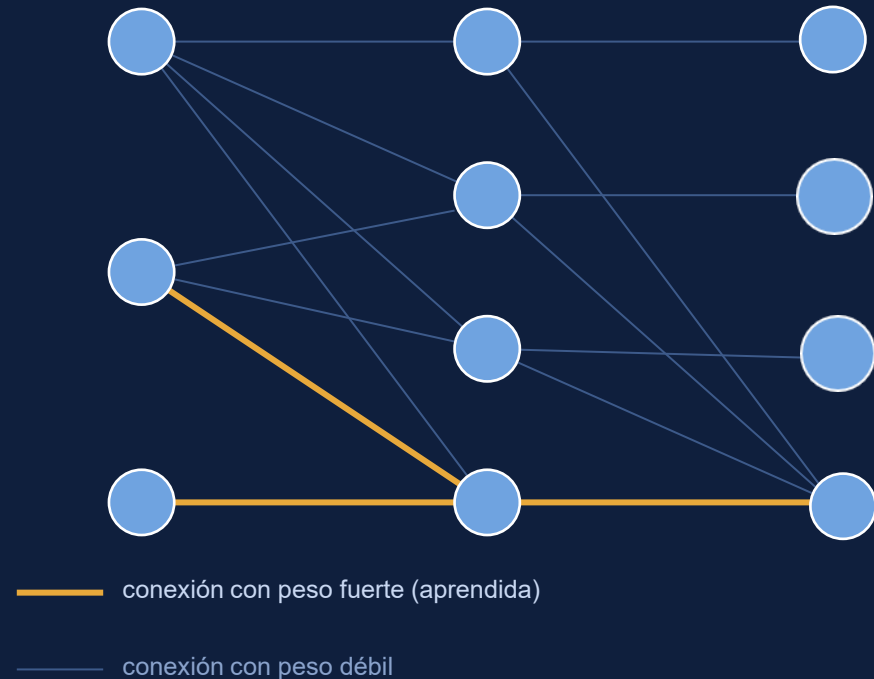
El modelo es la red más todo lo que aprendió: ya no es solo estructura, es conocimiento aplicable.

4

Ejemplo: un LLM

Un LLM como Gemini es un modelo entrenado específicamente con texto, listo para generar respuestas.

Red entrenada = Modelo



Los pesos (grosor de las líneas) se ajustan durante el entrenamiento

Usando roles del sistema

Cada mensaje tiene un role (quién habla) y un content (qué dice)



`"role": "system"`

System Role

Establece las reglas de comportamiento, restricciones, tono y la personalidad base del modelo antes de procesar el prompt.

```
"content": "Eres un tutor  
experto en Python..."
```



`"role": "user"`

User Role

Representa la entrada enviada directamente por el usuario (instrucciones, preguntas, documentos o consultas puntuales).

```
"content": "¿Qué es la  
Inteligencia Artificial?"
```



`"role": "assistant"`

Assistant Role

Contiene la respuesta generada por el modelo. La API se la asigna automáticamente; permite mantener el historial de la conversación.

```
"content": "La IA es la  
capacidad de..."
```

Parámetros de Configuración

Controlan cómo genera texto el modelo, no qué sabe



`temperature`

Temperature

Controla la aleatoriedad de la respuesta. Valores bajos (0.0–0.3) dan respuestas más predecibles y enfocadas; valores altos (0.7–1.0) dan respuestas más creativas y variadas.

```
temperature=0.1 # preciso  
temperature=0.9 # creativo
```



`max_tokens`

Max Tokens

Límite máximo de tokens que puede generar el modelo en su respuesta. No afecta el input, solo corta la salida si se excede.

```
max_tokens=500  
# corta la respuesta ahí
```

Modelo y Estructura de Costos

Cómo se calcula lo que cuesta usar un LLM por API



Proveedor vs Creador

Creador: la empresa que entrena el modelo (ej: Google entrena Gemini, Meta entrena Llama)
Proveedor: quien entrega el acceso vía API (ej: Google AI Studio, Groq, OpenRouter)



Métrica de Consumo por Tokens

Los costos se calculan por millón de tokens procesados (1 token $\approx$ 4 caracteres / 0.75 palabras en español/inglés).

Tipo de Token	Descripción	Objeto Response (OpenAI Spec)
Entrada (Prompt Tokens)	Tokens enviados en system message, prompt e historial.	<code>response.usage.prompt_tokens</code>
Salida (Completion Tokens)	Tokens generados por el modelo en la respuesta.	<code>response.usage.completion_tokens</code>
Total de Tokens	Suma de entrada + salida (determina costo final del call).	<code>response.usage.total_tokens</code>

💡 Los precios por token cambian con frecuencia — revisa la tarifa vigente en ai.google.dev/gemini-api/docs/pricing .

Antes de programar: capa gratuita y API key

Cómo ubicar esto tú mismo en la documentación de cualquier proveedor

1

Busca la capa gratuita

En la doc oficial del proveedor, busca "pricing" o "free tier". Ahí ves los límites (requests/día, tokens/minuto) y qué modelos son gratis.

```
ai.google.dev/gemini-api  
/docs/pricing
```

2

Obtén la URL del endpoint

Busca "API reference" o "OpenAI compatibility" en la doc. Ahí está el `base_url` correcto para conectar tu cliente.

```
ai.google.dev/gemini-api  
/docs/openai
```

3

Genera y guarda tu API key

Ve a la consola del proveedor, genera la key, y guárdala como variable de entorno. Nunca la escribas directo en el código.

```
aistudio.google.com  
/api-keys  
  
GEMINI_API_KEY=...
```




Ahora vamos al notebook a practicar

`1-gemini_model_api.ipynb`